ЛАБОРАТОРНАЯ РАБОТА · MNIST · ГРАФОВЫЕ НЕЙРОСЕТИ

Какой граф нужен GNN

Зафиксировали всё, кроме графа, перебрали 22 варианта связей между пикселями и посмотрели, что будет с точностью.

22 варианта графа **5** запусков каждого **110** обучений

ЧТО ДЕЛАЛИ

Меняли только связи между пикселями

Каждый из 784 пикселей картинки — вершина графа. Ребро означает, что два пикселя обмениваются информацией. Модель, число слоёв, скорость обучения, разбиение данных — всё одинаковое во всех 110 запусках. Отличается только то, *какие пиксели соединены*.

Пиксели заранее перемешаны, одинаково для всей выборки. Это нужно, чтобы граф нельзя было построить по координатам: модель не знает, где какая вершина на картинке.

ЕСЛИ НЕ ЗНАКОМ С GNN

Свёртка, у которой соседей задаёшь ты

Свёрточный слой в CNN смешивает пиксель с восемью соседями по сетке. Кто чей сосед — вшито в архитектуру и поменять нельзя: сетка есть сетка.

Графовый слой делает ровно то же смешивание, но список соседей приходит извне, отдельной таблицей: *матрицей смежности*. Строка таблицы — вершина, единица на пересечении — «эти двое обмениваются значениями».

Что происходит внутри слоя, по шагам

Возьмём один пиксель. Пусть его значение 0.7, а у четырёх его соседей по решётке — 0.2, 0.9, 0.1 и 0.6.

1. **Усреднение с соседями.** Складываем значение самой вершины и её соседей, делим на их число: `(0.7+0.2+0.9+0.1+0.6)/5 = 0.5`. Своё значение входит в сумму — иначе вершина забывала бы саму себя. Это единственное место, где вообще используется граф: он говорит, кого складывать.
2. **Умножение на обучаемые веса.** Полученное число 0.5 умножается на вектор весов длины 64 — получается 64 числа вместо одного. Это те самые параметры, которые учит градиентный спуск. На следующих слоях у вершины уже 64 числа, и веса становятся матрицей 64×64.
3. **ReLU.** Отрицательные значения обнуляются, как в обычной сети.

После этого у каждой из 784 вершин лежит вектор из 64 чисел. Слой закончен.

Почему «три раунда обмена»

После первого слоя вершина знает про себя и своих непосредственных соседей. Но её соседи на том же слое узнали про *своих* соседей. Поэтому на втором слое, усредняясь с ними, вершина получает информацию с расстояния два. На третьем — с расстояния три.

Число слоёв — это дальность, на которую информация расходится по графу. Три слоя = три шага, дальше вершина не видит ничего. Запомни это место: в конце отчёта оно окажется важнее, чем выглядит сейчас.

В самом конце все 784 вектора усредняются в один, он идёт в обычный линейный слой на 10 классов. Это усреднение называется readout, и из-за него порядок вершин не имеет значения — важный момент для следующего раздела.

То есть матрица смежности играет в GNN ту же роль, что сетка в CNN. Только её можно подменить. Вот мы и подменяли — 22 раза.

Почему у лучшего графа 0.53, а не 0.98

Первое, обо что спотыкаются: MNIST — это же решённая задача, откуда 53%? Ответ в том, что **модель не видит картинку**. Она видит 784 вершины, у каждой ровно одно число, и никаких координат, номеров или порядка.

Про «одно число» — MNIST чёрно-белый. Не фотография, а рукописная цифра: 28×28 точек, в каждой одно значение от 0 до 255. Ноль — чёрный фон, 255 — белые чернила, между ними серое. Мы делим на 255, получается от 0 до 1: *насколько эта точка закрашена*. У цветной картинки было бы три числа на пиксель (R, G, B), и у вершины был бы вектор из трёх — но у нас именно одно. Для масштаба: 81% всех пикселей MNIST — ровно ноль, пустой фон.

Сравни с обычным MLP. У него на каждый пиксель своя строчка весов, то есть он знает, что «пиксель №451» — это конкретный пиксель, всегда один и тот же. Отсюда его 0.98.

У графовой сети такого нет по построению: слой перестановочно эквивариантен, а в конце все вершины усредняются. Перенумеруй вершины вместе с графом — ответ не изменится ни на бит. Значит отличить вершину от вершины модель может **только по тому, как та связана с остальными**.

> **Отсюда и весь смысл работы**
>
> Граф — единственный источник различий между вершинами. Плохой граф не просто «чуть хуже»: он оставляет модель без информации о том, кто есть кто. Поэтому шкала и получается такой широкой, от 0.23 до 0.53.

Две границы шкалы предсказаны заранее — и сошлись

Прежде чем что-то запускать, две точки шкалы считаются на бумаге.

Граф без рёбер. Вершины ни с кем не общаются, усреднять не с кем. Модель получает 784 числа без всякой привязки к тому, где какое находится. Всё, что из них извлекается, — *сколько точек какой яркости встретилось*: столько-

то чёрных, столько-то серых, столько-то белых. Гистограмма значений, без позиций.

Полный граф. Каждая вершина усредняется со всеми 783 остальными, то есть все получают одно и то же число — среднее по картинке. После первого же слоя вершины неразличимы, и модель может опираться только на *среднюю закрашенность*: сколько всего чернил ушло на цифру.

Это правда немного разделяет классы. Единица — самая «худая» цифра, у неё закрашено в среднем 7.6% площади; ноль самый «жирный», 17.3%. Но спутать по такому признаку можно почти всё.

> **Проверка, не зависящая от нашего кода**
>
> Оба предсказания проверяются в обход графовой сети целиком: обучить обычную логистическую регрессию ровно на тех признаках, к которым, по нашей теории, сводится модель.
>
> | ЧТО ПОДАЛИ НА ВХОД | РЕГРЕССИЯ | НАШ GNN |
> |---|---|---|
> | только средняя закрашенность, 1 число | `0.2135` | `0.2299` complete |
> | только гистограмма яркостей, 16 корзин | `0.3162` | `0.3081` empty |
>
> Совпадение в пределах пункта, причём считалось совсем другим кодом. Это и был первый тест всей работы: не сойдись границы шкалы там, где предсказано, дальше идти было бы нельзя — значит где-то ошибка, и все 110 запусков мерили бы её.

Если ты видел ChebNet и удивился цифре

В классической статье Defferrard 2016 пиксели MNIST тоже вершины графа, и точность там около 99%. Разница не в удаче: там модели *возвращают* знание о том, где какая вершина. Фильтры богаче, есть огрубление графа, а в конце полносвязная голова, работающая с вершинами по их номерам.

Мы это выключили намеренно. Как только у модели появляется способ узнать вершину в обход графа, эксперимент теряет смысл: непонятно, что даёт граф, а что — номера. Поэтому здесь нет ни позиционных кодировок, ни номеров вершин, а readout — простое среднее. **Низкая точность тут не недостаток, а условие задачи.**

ОТКУДА 22 ВАРИАНТА

Каждая группа отвечает на свой вопрос

Графы не набирались наугад. Они разбиты на группы, и каждая закрывает отдельное возражение к выводам.

Оракулы — знают правильный ответ

`grid4`: пиксель соединён с четырьмя соседями — сверху, снизу, слева, справа. Настоящая решётка 28×28. Строится по координатам, которые модели неизвестны, поэтому это не метод, а *верхняя отметка по структуре*: столько даёт знание правильного ответа.

`grid8`: то же плюс четыре диагонали. Вдвое больше рёбер, и появляются треугольники — у решётки из четырёх соседей их нет вообще.

Честные — строятся только по данным

`knn_corr_k8`: для каждого пикселя считаем корреляцию его яркости с яркостью каждого другого пикселя по 55 000 обучающих картинок. Берём

восемь самых скоррелированных и проводим рёбра. Координат метод не знает вовсе — только то, какие пиксели зажигаются вместе.

Получается похоже на решётку, но не решётка: соседние пиксели правда коррелируют (чернила непрерывны), зато метод дополнительно связывает далёкие места, которые срабатывают одновременно. Степени выходят неровные — от 0 до 16 при средней 8.8.

`thr_corr`: тот же принцип, но вместо «по восемь на каждого» берём глобальный порог на корреляцию, подобранный так, чтобы рёбер вышло ровно столько же, сколько у `grid4`. Честное сравнение по числу связей, но структура совсем другая: у кого-то 12 соседей, у кого-то ни одного.

> **Мёртвые пиксели**
>
> 69 пикселей по краям картинки чёрные *на всех* обучающих примерах. У постоянной величины корреляция не определена — делить на нулевой разброс нельзя. Такие пиксели остаются одинокими вершинами без единого ребра, и потому у всех honest-графов не одна связная компонента, а 70. У `thr_corr` и вовсе 152 одиночки. Это не баг, это свойство данных, и оно честно попадает в таблицу.

Контроли — те же рёбра, но случайные

Без них любое «граф X помогает» ничего не значит: может, дело просто в количестве связей. Правило было жёстким — ни одно утверждение не принимается без контроля, согласованного по плотности.

`er_grid4`: берём ровно 1512 случайных пар вершин, как у решётки. Никакой структуры.

`regular4`: случайный граф, но у *каждой* вершины ровно по 4 соседа. Отвечает на возражение «может, дело в том, что у решётки степени ровные».

`config_knn8`: берём распределение степеней от `knn_corr_k8` — кто сколько имел соседей — и перетасовываем рёбра случайно, сохраняя эти числа.

Отвечает на «может, дело в том, что у части пикселей связей много, а у части мало».

`grid4_misaligned` — **главный контроль всей работы**. Берём решётку и переклеиваем её к вершинам по случайной перестановке. Рёбра те же, узор тот же, поменялось только то, какой пиксель сидит в какой вершине.

Кривая деградации — плавный переход

`ws_p005` … `ws_p1`: берём решётку и переставляем долю рёбер — 5%, 10%, 25%, 50%, 100%. Перестановка двойная, парами, поэтому число соседей у каждой вершины сохраняется *точно*: было 4, осталось 4. Меняется ровно одно — насколько связи локальны.

Это не точка, а кривая, и она самая наглядная во всей таблице: 0.43 → 0.41 → 0.37 → 0.32 → 0.30 по мере разрушения локальности.

Патологии — задают нижнюю часть шкалы

`star`: одна центральная вершина соединена со всеми остальными 783, больше рёбер нет. `path`: все 784 вершины в одну длинную цепочку. `blocks`: кластеризуем пиксели по корреляции, внутри каждого кластера соединяем всех со всеми, между кластерами — пусто. Нужны, чтобы шкалу задавали и снизу, и чтобы не было соблазна выкинуть неудобного кандидата.

Как они выглядят в числах

ГРАФ	РЁБЕР	СОСЕДЕЙ В СРЕДНЕМ	РАЗБРОС СТЕПЕНЕЙ	ОДИНОКИХ ВЕРШИН	ТРЕУГОЛЬ-НИКОВ	ДИАМЕТР
grid4	1512	3.9	0.36	0	0.000	54
grid4_misaligned	1512	3.9	0.36	0	0.000	54
er_grid4	1512	3.9	1.93	17	0.003	10
grid8	2970	7.6	1.07	0	0.454	27
knn_corr_k8	3456	8.8	3.16	69	0.440	13
thr_corr	1512	3.9	3.00	152	0.204	34

ГРАФ	РЁБЕР	СОСЕДЕЙ В СРЕДНЕМ	РАЗБРОС СТЕПЕНЕЙ	ОДИНОКИХ ВЕРШИН	ТРЕУГОЛЬ-НИКОВ	ДИАМЕТР
regular4	1568	4.0	0.00	0	0.006	8
star	783	2.0	27.9	0	0.000	2
path	783	2.0	0.05	0	0.000	783
blocks	51762	132.0	125.0	0	1.000	1

«Разброс степеней» — насколько неровно распределены соседи: 0 значит у всех поровну. «Треугольников» — доля замкнутых троек, у решётки их нет вообще, у клик единица. «Диаметр» — сколько шагов между двумя самыми далёкими вершинами.

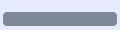
> ### Две вещи, которые видно прямо из таблицы
>
> **Первые две строки совпадают полностью.** `grid4` и `grid4_misaligned` — по всем шести колонкам одно и то же, включая диаметр 54 и полное отсутствие треугольников. Это не совпадение, а следствие того, что второй получен из первого перенумерацией. А точность у них 0.49 и 0.30.
> **А вот `er_grid4` — другой граф.** Рёбер столько же, но диаметр 10 вместо 54, степени разбросаны впятеро сильнее, появились одинокие вершины. Поэтому одного случайного контроля мало: он отличается от решётки и по структуре тоже, и непонятно, что именно сработало. Изоморфная пара такой лазейки не оставляет.

РЕЗУЛЬТАТЫ

Все 22 варианта, по убыванию точности

Синие полоски — графы, соединяющие похожие пиксели, серые — случайные, фиолетовые — промежуточные. Верхняя строка `mlp` — обычная нейросеть без графа, она стоит здесь как верхняя отметка шкалы.

ГРАФ	ЧТО ЭТО	ACCURACY	MACRO-F1	
mlp	обычная нейросеть без графа	0.9840 ±0.0009	0.9838	
grid8	решётка: пиксель и 8 соседей	0.5303 ±0.0161	0.4987	
knn_corr_k16	16 самых похожих пикселей	0.5243 ±0.0155	0.5064	

ГРАФ	ЧТО ЭТО	ACCURACY	MACRO-F1	
knn_corr_k8	8 самых похожих пикселей	0.5202 ±0.0065	0.4962	
thr_corr	самые похожие пары пикселей	0.5051 ±0.0142	0.4778	
grid4	решётка: пиксель и 4 соседа	0.4942 ±0.0102	0.4583	
knn_corr_k4	4 самых похожих пикселя	0.4855 ±0.0158	0.4584	
ws_p005	решётка, 5% рёбер переставлено	0.4259 ±0.0149	0.3968	
ws_p01	решётка, 10% рёбер переставлено	0.4147 ±0.0114	0.3780	
blocks	кластеры похожих пикселей	0.3842 ±0.0018	0.3689	
ws_p025	решётка, 25% рёбер переставлено	0.3712 ±0.0058	0.3372	
er_grid4	случайный граф, рёбер как у решётки	0.3314 ±0.0207	0.2846	
path	вершины в одну цепочку	0.3199 ±0.0066	0.2795	
ws_p05	решётка, 50% рёбер переставлено	0.3193 ±0.0139	0.2842	
star	все вершины к одной центральной	0.3128 ±0.0201	0.2609	
empty	граф без рёбер	0.3081 ±0.0029	0.2639	
ws_p1	решётка, 100% рёбер переставлено	0.3047 ±0.0141	0.2604	
grid4_misaligned	решётка, приклеенная к случайным пикселям	0.2988 ±0.0095	0.2487	
regular4	случайный граф, у всех по 4 соседа	0.2927 ±0.0085	0.2503	
config_knn8	случайный граф с теми же степенями	0.2815 ±0.0044	0.2272	
er_knn8	случайный граф, рёбер как у knn_corr_k8	0.2756 ±0.0038	0.2238	

ГРАФ	ЧТО ЭТО	ACCURACY	MACRO-F1	
`complete`	все вершины соединены со всеми	`0.2299` ±0.0017	`0.1594`	

ЧТО ПОЛУЧИЛОСЬ

Три вывода

1. Граф решает очень много

От `0.23` до `0.53` при полностью одинаковой модели.

2. Важно не количество рёбер, а какие они

Случайный граф с тем же числом рёбер, что у решётки, даёт `0.33` — не лучше, чем вообще без графа (`0.31`).

3. Слишком много связей хуже, чем ничего

Полный граф — `0.23`, худший результат. Когда каждая вершина слушает всех, они усредняются и становятся неразличимы: остаётся только средняя яркость картинки.

САМОЕ НАГЛЯДНОЕ

Один и тот же граф, разница в 20 пунктов

0.4942

`grid4`

Решётка, подключённая правильно: соседи по графу — соседи по картинке.

0.2988

`grid4_misaligned`

Та же решётка, подключённая к пикселям в случайном порядке.

Второй граф получен из первого одной операцией: вершины переставили случайным образом, а рёбра оставили те же. Как если бы схему рассадки в классе не трогали, но студентов рассадили в другом порядке. Схема та же — класс работает иначе.

Совпадает у них **всё, что можно посчитать по одной матрице смежности**: 1512 рёбер, у каждой вершины ровно столько же соседей, та же связность, тот же диаметр, тот же спектр, тот же коэффициент кластеризации. Не «примерно те же» — побитово те же, и это гарантировано самой конструкцией, а не совпадением.

А точность отличается на 19.5 пункта. Разрыв огромный по меркам статистики: t-тест даёт $p = 1.2 \cdot 10^{-9}$ при пяти запусках, размер эффекта почти двадцать сигм. Это не шум.

Почему это важнее, чем кажется

Хочется найти формулу «хорошести» графа: посчитать что-нибудь по матрице смежности и заранее узнать, сработает ли этот граф. Желание понятное — иначе приходится обучать по модели на каждый вариант, а это часы. Кандидатов в такую формулу много: спектральная щель, доля треугольников, разброс степеней, диаметр.
Шаг первый. Любая такая формула по определению берёт на вход *только матрицу смежности*. Больше ей ничего не дано — ни картинок, ни того, какой пиксель где.
Шаг второй. Перенумеровать вершины — значит записать тот же самый граф по-другому. Как переставить строки и столбцы в таблице: содержимое не изменилось, изменилась запись. Поэтому любая формула, считающая «свойство графа», обязана давать на обеих записях *одно и то же число*. Иначе она мерила бы не граф, а порядок строк.
Шаг третий. У нас есть ровно такая пара: `grid4` и `grid4_misaligned` — один граф в двух записях. Значит любая формула выдаёт на них одинаковое значение. Мы это и видим в таблице выше: все шесть колонок совпадают до последнего знака.
Шаг четвёртый. А качество разное: 0.49 и 0.30.

Получается, что на одном и том же входе нужны два разных выхода. Функция так не умеет — это противоречие, а не сложность. Следовательно **формулы, предсказывающей качество по одному только графу, не существует**. Не «мы её пока не нашли» — её не может быть, и это доказано одним примером, как контрпример опровергает теорему.

Что делать вместо этого. Смотреть надо не на граф, а на *пару «граф и данные»*. Вопрос «соединяет ли это ребро похожие пиксели?» по одной матрице смежности не решается — нужны ещё сами картинки. Такая величина уже не свойство графа, а свойство их совместимости, и на неё запрет не распространяется.

Мы такую величину и попробовали — гладкость сигнала на графе. Как критерий выбора она не сработала, об этом ниже. Но искать надо именно там: направление «померить один только граф» закрыто окончательно.

Есть и побочное наблюдение, которое стоит отдельного внимания. `knn_corr_k8` — граф, построенный на одних корреляциях по обучающей выборке, ничего не знающий про геометрию картинки — дал `0.52` против `0.49` у настоящей решётки. Честный метод обошёл оракула.

То есть решётка не оптимальна для графовой сети даже тогда, когда геометрия известна точно. **Почему — мы не знаем.** Одно объяснение проверили и отвергли, о нём ниже.

Что проверяли и отвергли

Два объяснения выглядели убедительно и не пережили проверку. Обоим место здесь, а не в столе: они очерчивают границу того, что работа утверждает.

Гладкость сигнала как критерий выбора графа — нет

Ожидалось, что качество предсказывается энергией Дирихле: свёртка работает как фильтр низких частот, значит хорош граф, на котором соседи держат похожие значения. По всем 22 кандидатам корреляция сильная, −0.795. Но держится она целиком на вырожденных графах: стоит оставить шесть осмысленных, и знак **переворачивается на +0.543**. То есть величина отличает граф от мусора, но выбрать лучший из двух нормальных графов ею нельзя — а нужно именно это.

Симметрия решётки как причина ошибок — нет

У решётки 28×28 восемь симметрий, и GNN на ней математически обязан давать одинаковый ответ на цифру и на неё же, повёрнутую на 180°. Теорема верна и проверена точно: ответ совпадает побитово. Казалось, отсюда и путаница 6 с 9, которой у решётки в 64 раза больше, чем у обычного MLP.

Проверка прямым контролем это отвергла. Модель, *насильно* сделанная точно инвариантной к этим восьми симметриям, даёт путаницу 1.39 — вдвое меньше, чем 3.21 у решётки. И ошибается она на других парах: 2 с 5, а не 6 с 9. Механизм даёт меньше половины эффекта и бьёт не туда. Цена

самой инвариантности при этом измерена и невелика: 2.07 пункта на MNIST.

Общее у обоих провалов: гипотезу придумывали, уже глядя в таблицу. Отсюда правило на дальнейшее — формулировка и критерий подтверждения фиксируются письменно до того, как посмотришь в данные.

Что стоит оговорить

Обучали 150 эпох, и у 15 кандидатов из 22 лучший результат пришёлся на эпохи 136–146 — впритык к концу. При более долгом обучении их числа были бы выше, так что это нижние оценки. На сравнение не влияет: бюджет одинаковый у всех.

Все числа получены при **трёх слоях**, и это не нейтральный выбор. За три шага решётка видит 23 пикселя из 784, а плотный случайный граф — 462. Разным графам нужна разная глубина, чтобы охватить картинку: решётке 31 слой, случайному 4. Так что таблица сравнивает графы в режиме, который одним подходит, а другим нет. Это следующее, что проверяется.

СВЯЗЬ С РАБОТОЙ ЛАБОРАТОРИИ

Тот же вопрос, что в статье про SSONN

В статье лаборатории про SSONN сеть наращивается локально: по градиентам выбираются отдельные рёбра вычислительного графа, и в эти места вставляются новые нейроны. Главный вопрос там — *правда ли критерий выбирает нужные рёбра*, или сгодились бы любые.

Наша лабораторная проверяет ровно это, только в чистом виде: архитектура зафиксирована, меняется только набор рёбер. Ответ однозначный — выбор рёбер решает почти всё. Случайный набор того же размера даёт `0.33`, то есть не лучше, чем вообще без рёбер (`0.31`).

Графы при этом разные по смыслу: у нас — над пикселями, в статье — над вычислениями сети. Числа между работами не переносятся, совпадает только способ проверки.

ПРОДОЛЖЕНИЕ

Проверка на данных, где правильный ответ известен

У всего выше есть общая слабость: на MNIST истинного графа не существует. Решётка — разумное предположение, но не факт, и граф по корреляциям её обходит. Значит фразу «метод нашёл структуру» там можно сказать только приблизительно.

Поэтому был сделан генератор, где граф задаём мы сами. Класс объекта — *число связных пятен* активных вершин. Хитрость в том, что в каждом объекте ровно 24 активные вершины со значением 1, независимо от класса: **гистограмма значений одинакова по построению**, и различить классы можно исключительно по расположению на графе.

0.669
истинный граф

0.243
случайный и пустой
случайное угадывание на четырёх классах — 0.250

Без графа — ровно угадывание, как и предсказано конструкцией. Это чище, чем MNIST, где даже пустой граф даёт 0.31 за счёт гистограммы яркостей.

Сколько рёбер настоящего графа находят разные методы

Здесь это измеримо точно, и без всякого обучения сети.

СПОСОБ	ТОЧНОСТЬ	ПОЛНОТА	F1	УСТОЙЧИВОСТЬ
knn_corr_k4	0.849	1.000	0.918	0.915
thr_corr	0.875	0.875	0.875	0.754
emb_learned_k4	0.672	0.909	0.773	0.689

СПОСОБ	ТОЧНОСТЬ	ПОЛНОТА	F1	УСТОЙЧИВОСТЬ
`grad_sim_k4`	0.244	0.394	0.301	0.116
`случайный граф`	0.026	0.061	0.036	—

«Устойчивость» — насколько совпадают графы, построенные по двум непересекающимся половинам выборки.

> **Простая корреляция выигрывает у всего остального**
>
> Обычный kNN по корреляциям находит **все** настоящие рёбра при точности 0.85 — почти решает задачу восстановления структуры. Два более изощрённых метода ему проиграли.
> **Обучаемые эмбеддинги признаков** дали 0.773 против 0.918. Ожидание было обратным: низкий ранг должен был давить ложные корреляции редких признаков.
> **Граф по градиентам модели** провалился глубоко: полнота 0.394, то есть он не находит и половины настоящих связей. Причину видно по устойчивости 0.116 — на двух половинах данных метод даёт почти непересекающиеся графы, то есть ловит шум обучения, а не структуру.

Наращивать число соседей вредно

Полнота упирается в единицу уже при четырёх соседях — все настоящие рёбра найдены. Дальше растёт исключительно число ложных:

`k=4 → F1 0.918` · `k=8 → 0.601` · `k=16 → 0.345`

Одинаково у всех методов. Значит число соседей надо выбирать по точности, а не наращивать. Оговорка: истинный граф здесь — решётка со средней степенью 3.7, поэтому четвёрка попадает в неё почти точно; на графах с другой степенью надо перепроверять.

Метрика устойчивости оказалась с подвохом

Первый прогон выдал случайным графам устойчивость 0.997 и 1.000 — почти идеальную. Причина очевидна задним числом: они вообще не смотрят на данные, строятся от фиксированного числа, и от смены половины выборки не меняются.

То есть метрика в наивном виде **награждает за игнорирование данных**. Читать её можно только вместе с восстановлением: высокая устойчивость при низком F1 значит, что метод стабильно находит не то. Именно в этой паре она и разоблачила граф по градиентам.

ЧТО ДАЛЬШЕ

Три вопроса, которые остались открытыми

Всё выше — про фиксированную сеть в три слоя. Это был осознанный выбор: что-то надо было закрепить, чтобы менять только граф. Но теперь видно, что сам этот выбор не нейтрален.

Три слоя — почему именно три?

За три раунда обмена информация проходит по графу три шага. У решётки это 23 пикселя из 784 — три процента картинки. У плотного случайного графа за те же три шага — 462, больше половины. Чтобы охватить всю картинку, решётке нужен 31 слой, а плотному графу четыре. Разница почти в десять раз. Значит любое сравнение при фиксированной глубине сравнивает не графы, а их совместимость с этой конкретной глубиной. *Наша собственная таблица под ту же претензию и попадает.*

Всё меряно на одной архитектуре

Графовая свёртка усредняет соседей — она работает как фильтр низких частот, ей нужен граф, на котором соседи похожи. У других архитектур логика другая: GAT сам взвешивает рёбра вниманием, GIN суммирует вместо усреднения. Вполне может оказаться, что «хороший граф» — понятие относительное и зависит от того, что за сеть сверху.

Почему честный граф обошёл решётку

Самая интересная незакрытая аномалия. Одно объяснение — через симметрии решётки — проверили прямым контролем и отвергли: оно даёт меньше половины наблюдаемого эффекта и бьёт по другим парам цифр. Настоящей причины пока нет.

Под каждый вопрос уже написан код и заранее зафиксированы критерии — что именно будет считаться подтверждением, и на каком подмножестве графов. Заранее, до запусков: два красивых объяснения в этой работе уже развалились именно потому, что придумывались после того, как посмотрели в таблицу.

MNIST, 55 000 обучающих объектов, 10 000 тестовых. Во всех запусках одна модель: трёхслойная графовая свёрточная сеть, 64 скрытых канала, усреднение по вершинам, Adam с шагом 0.001. Каждый вариант графа обучен пять раз с разной инициализацией; в таблице среднее и разброс. Графы, строящиеся по данным, считались только на обучающей выборке.